

Your grade: 100%

Your latest: 100% • Your highest: 100% • To pass you need at least 80%. We keep your highest score.

Next item →

1 / 1 point

1. What is one benefit of using linked lists over arrays?

- ☐ Linked lists automatically prevent memory leaks.
- ☒ Linked lists allow flexible insertion and deletion of elements.
- ☐ Linked lists have fixed memory allocation.
- ☐ Linked lists are easier to sort than arrays.

✔ Correct
One of the main benefits of linked lists is their flexibility in inserting and deleting elements without the need for shifting other elements, unlike in arrays.

1 / 1 point

2. Select each of the following statements about linked lists that is true.

- ☒ Linked lists are more efficient than arrays when inserting a new item, especially to a location in the middle of the linked list

✔ Correct
Inserting an item to the middle of an array requires shifting over every following item, leading to significant inefficiencies. Insertion with a linked list, on the other hand, requires traversal to the desired location, but then only requires updating the pointers of the nodes at the point of insertion.

- ☐ The items in a linked list must be sorted for the linked list to operate correctly
- ☐ Linked lists are more efficient than arrays for accessing individual items
- ☒ Linked lists require additional memory to store a pointer with each item that points to the next item

✔ Correct
The pointer included with each element in the node of a linked list does in fact require additional memory.

1 / 1 point

3. What is a key advantage of using arrays in Python?

- ☐ Arrays are an efficient data structure when you will frequently be inserting and deleting items
- ☐ Arrays can store even billions of items without performance or memory concerns
- ☐ Arrays automatically sort the data they contain
- ☒ Arrays allow quick access and modification if you know the index of the item to be accessed

✔ Correct
One of the main advantages of arrays is their ability to allow quick access and modification of elements when the index is known, making them efficient for certain operations.

1 / 1 point

4. Select each of the following true statements on working with an LLM to choose and implement data structures.

- ☒ Assigning the LLM an expert role can allow you to profile the data structure decisions you make from a particular perspective

✔ Correct
As shown in the lectures, assigning an expert role like this is a good practice in many instances, including when implementing data structures.

- ☒ LLMs can generate code to implement a certain data structure in the context of your project

✔ Correct
LLMs are great at generating sample code to show you how to implement a given data structure

- ☐ Only beginner programmers are likely to benefit from working with an LLM to implement data structures in their programs

- ☐ LLMs can provide generic information on data structures but can't relate that information to the context of your specific projects

1 / 1 point

5. Consider the following code that uses the Node and LinkedList classes covered in the lectures. What will be printed when this code is run?

```
1 import threading
2
3 class Node:
4     def __init__(self, data):
5         self.data = data
6         self.next = None
7
8 class LinkedList:
9     def __init__(self, max_size=None):
10        self.head = None
11        self.size = 0
12        self.max_size = max_size
13        self.lock = threading.Lock()
14
15    def append(self, data):
16        # Validate input data
17        if len(data) > 1000:
18            raise ValueError("Data size exceeds maximum limit")
19        with self.lock:
20            if self.max_size is not None and self.size >= self.max_size:
21                raise ValueError("Linked list is full")
22            new_node = Node(data)
23            if self.head is None:
24                self.head = new_node
25            else:
26                last = self.head
27                while last.next:
28                    last = last.next
29                last.next = new_node
30            self.size += 1
31
32    def print_list(self):
33        current = self.head
34        while current:
35            print(current.data, end=" ")
36            current = current.next
37
38 mylist = LinkedList()
39 mylist.append("X")
40 mylist.append("X")
```

- ☐ XYZ
- ☒ XXYZ
- ☐ ZYX
- ☐ ZYXX

✔ Correct
The four items are printed in the same order they were added in